DSA: Assignment - 2

Abstract

In this assignment you will handle different types of data structures like arrays, dictionaries, and heaps.

1 Tasks

You are allowed to use the available python builtin data structures (i.e. list, dict, tuple etc.). You are also allowed to use the builtin `sorted()` list sorting function and `len()`.

⚠**Do not change function names or else pytest will not work.** In order to test your implementation the function names are hardcoded in the automated testing environment. **Do not change the function names in the boiler plates**. If you change the function names `pytest` will not work, you will not be able to hand-in your assignment and your submission will result in **0 pt**.

⚠**Be careful using global variables when working on your assignments.** In order to test your implementation `pytest` calls your functions multiple times with different parameters but in the same runtime. Make sure to initialize any global variable you use appropriately (e.g. in the beginning of the sort function).

1.1 Task 1: Findings risky neighbourhoods (3pt)

The water management department has a network of surveillance sensors at every corner of the Amsterdam sewage system. Every sensor reports the level of chemical waste in the water. Each day the sensors are queried for an updated measurement roughly by neighbourhood. Therefore, the measurements from sensors that are in close proximity to each other are initially stored on consecutive array locations.

The water management department takes in water from a few different random neighborhoods at a time for treatment, but has to make sure the level of chemical waste remains below a certain level. If the level is too high, nature and people will suffer.

You are tasked with implementing an algorithm that finds the smallest subsequence in the readings (positive integers) whose sum of elements exceed the threshold defined by the water management department. If no such subsequence exists your algorithm should return 0.

Example 1. *The* `smallest_subsequence_length` *function takes two parameters:*

- `array` *(`list[int]`): An array of positive integers*
- `k` *(`int`)*

The function returns a the length of the smallest subsequence as an Integer.

Call: `smallest_subsequence_length([1, 2, 3, 4, 5, 6, 7, 8], 20)`

Returns (int): `3`

1.2 Task 2: Malfunctioning sensors in De Pijp (3pt)

Unfortunately some of the sensors started to malfunction. Presumably, they report the reading value more than once each day. The water management department has pinned it down to the neighbourhood De Pijp, and a few suspicious measurements have been identified for you. Since there is some time pressure to fix the problem, your colleague has already sorted the measurements for you late last night. Your task is to implement an algorithm that counts occurrences of a given number. Of course, the array may contain duplicated values and the algorithm should return 0 if it can not find any such occurrence.

Example 2. *The* `find_occurrences` *function takes two parameters:*

- `array` *(`list[int]`): An sorted array of positive integers*
- `n` *(`int`)*

The function returns an Integer representing the number of occurrences if found of `n` *in* `array`

Call: `find_occurrences([1, 1, 2, 3, 4, 4, 4, 8, 10], 1)`

Returns (int): `2`

1.3 Task 3: Checking for the right data structure (4pt)

The cooperation between the water management and health authorities has been very successful and thanks to your programming skills, the new covid measurement system is about to go online for every city in the Netherlands. However, after a long discussion with your colleague, you both realized that some of the new algorithms will not scale to the new amounts of readings you will have to handle. In particular, the new algorithms include an enormous amount of calls to a function called $findmin$. You decided to implement a new data structure, and your colleague did a first attempt last week. Since this is a huge opportunity, you both want to make sure the data structure was implemented correctly.

You task is to implement an algorithm that checks it a given integer array represents a min-heap or not.

Example 3. *The* `is_min_heap` *function takes one parameters:*

- `array` *(`list[int]`): An array of positive integers*

The function returns a Boolean or an Integer

Call: `is_min_heap([1, 2, 5, 3, 4, 6, 7])`

Returns (Union[int, bool]): `1 | True`